

∴ {cover}

RHEL 9 · RHCSA 实操讲义

02

标准输入输出、重定向与管道

从 FD 路由到管道状态：把数据去向、证据边界与失败判断放进同一条推理链。

对象模型操作语义验证诊断经典任务

大号阅读版

...

... { .reading-nav }

本章阅读导航

先抓住一条主线：Shell 不是在命令结束后搬运屏幕文字，而是在命令启动前建立文件描述符连接。学习时始终把“数据路径”和“退出状态”分成两条证据链。

01 识别数据源谁写 stdout、谁写 stderr、谁读取 stdin

02 标出 FD 为每个命令写出 0、1、2

03 建立管道先连接左 stdout 与右 stdin

04 应用重定向再从左到右改写各阶段端点

05 保留证据文件、终端与 tee 各证明一层

06 判断状态 \$?、PIPESTATUS 与 pipefail

专题地图 - **知识专题** 从终端表象回到文件描述符模型 - **操作专题** 构造可判定的 stdout、stderr 和退出状态 - **操作专题** 覆盖、追加与分别保存两个输出流 - **知识专题** `2>&1` 的复制语义与顺序差异 - **知识专题** 管道连接对象与 stderr 旁路 - **操作专题** 用 `tee` 在流经位置保留证据 - **操作专题** 用 `PIPESTATUS` 和 `pipefail` 判断失败 - **操作专题** here-document 的必要范围 - **诊断专题** 从表象回溯 FD 路由和阶段状态 - **经典任务** 分流证据；定位中间阶段失败

阅读时持续回答 1. 当前命令的 FD 0、1、2 分别指向哪里？ 2. 哪一步打开、截断、追加或复制了端点？ 3. 普通管道传递的是哪个流？ 4. stderr 是否仍沿原端点旁路？ 5. `tee` 文件证明数据到达了哪个位置？ 6. 哪个命令的退出状态被 \$? 保存？ 7. `PIPESTATUS` 是否已被后续命令覆盖？ 8. 当前证据不能证明什么？ ...

{ .nav-note } **前后章边界：** 第 01 章负责引用和展开；本章只轻量引用 here-document delimiter 的引用效果。`grep`、`sed`、`awk` 的处理语义留给第 05 章，Journal 与 rsyslog 留给第 13 章。 ...

...

第 02 章 · 正文

在终端里看到“文字出现在哪里”，很容易让人误以为 stdin 固定来自键盘、stdout 和 stderr 固定显示在屏幕上。实际上，命令只面对一组已经打开的文件描述符。Shell 先解析命令行；对于管道，它先建立相邻阶段的默认连接，再按每个命令从左到右应用显式重定向，最后启动各阶段。因此真正需要追踪的是“FD 编号当前指向哪个端点”，而不是屏幕上最后出现了什么。

本章围绕一条统一主线推进：

数据由谁产生

- 通过哪个 FD 离开或进入命令
- 管道先建立了哪些默认连接
- 显式重定向如何从左到右改写端点
- 哪些数据被保存，哪些仍然可见
- 每个阶段以什么状态结束
- 现有证据能证明什么，又不能证明什么

最常见的误判都可以从这条主线解释：命令主体没有成功，但 `>` 已经把旧文件清空；`2>&1` 顺序写反后，错误仍出现在终端；管道末端成功退出，却掩盖中间阶段失败；`tee` 文件有数据，却不能证明上游和下游都成功。第 01 章已经建立 Shell 解析和引用前提，本章只在 here-document 处轻量引用；文本筛选和日志系统分别留给后续章节。

核心概念

⋮ {concept-block}

概念

文件描述符（file descriptor）是进程访问已打开输入输出对象的编号。`0`、`1`、`2` 是常用入口，不是键盘、屏幕或某个文件本身；终端、普通文件、管道和 `/dev/null` 才是它们可能指向的端点。判断一条命令时，应写出“FD 编号 → 当前端点”，而不是把编号直接翻译成设备。⋮

⋮ {concept-block}

概念

标准输入、标准输出与标准错误是程序约定使用的三个数据通道，默认对应 FD `0`、`1`、`2`。stdout 通常承载可继续处理的数据，stderr 通常承载诊断信息，但这只是程序接口约定；文字是否“像错误”、终端是否着色，都不能替代对通道和退出状态的独立观察。⋮

⋮ {concept-block}

概念

重定向目标是 Shell 在命令启动前为某个 FD 建立的新端点。`>`、`>>` 和 `2>` 会打开文件，`2>&1` 会复制另一个描述符在该时刻的端点。重定向按从左到右应用，所以顺序不是排版差异，而是状态变化顺序；目标无法打开时，命令主体可能根本没有开始。⋮

⋮ {concept-block}

概念

管道（pipeline）是由 Shell 建立的一组相邻进程连接。普通 `A | B` 把 A 的 stdout 连接到 B 的 stdin，stderr 默认仍沿各自原端点输出。管道传递的是数据流，不是把左侧结果变成右侧命令参数；每个阶段仍是独立执行对象，并拥有独立退出状态。⋮

⋮ {concept-block}

概念

PIPESTATUS 是 Bash 保存最近一个前台管道各阶段退出状态的数组，元素顺序与管道从左到右一致。它回答“哪一段失败”，但会被后续简单命令更新，因此必须立即复制；数据文件存在、终端出现结果或单一 `$?`

都不能替代逐阶段状态。...

... {concept-block}

概念

`pipefail` 是当前 Bash 的管道汇总策略。默认情况下，管道状态来自最后一个阶段；启用 `pipefail` 后，汇总状态取最右侧非零阶段，全部为零时才为零。它让中间失败更容易暴露，却不会列出所有失败阶段，也不会自动保存数据或替代 `PIPESTATUS`。...

操作语义速查

... {quickref}

... {quickref-entry}

`printf` / `cat`

SYNOPSIS

```
printf FORMAT [ARGUMENT ...]  
cat [FILE ...]
```

用 `printf` 产生可预测的 stdout，用 `cat` 把文件或 stdin 原样送到 stdout，建立不受业务状态干扰的最小观察环境。

重要参数 / 形式

```
printf '%s\n' 'DATA:alpha'
```

明确产生一行 stdout，优先于依赖实现差异的 `echo`。

```
printf '%s\n' 'WARN:beta' >&2
```

让本次 `printf` 的 FD 1 复制 FD 2 当前端点，构造确定的 stderr。

```
cat file / cat < file
```

前者由 `cat` 打开参数文件；后者由 Shell 先把 FD 0 指向文件。

...

... {quickref-entry}

< / > / >>

SYNOPSIS

```
command < input  
command > output  
command >> output
```

分别为 stdin 选择文件、覆盖 stdout 目标、追加 stdout 目标。文件打开和截断发生在命令主体运行前。

重要参数 / 形式

`< input`

让 FD 0 从文件读取。

`> output / 1> output`

以覆盖方式打开 stdout 目标；既有内容可能先被截断。

`>> output / 1>> output`

在文件末尾追加，不清空旧内容。

...

... {[.quickref-entry](#)}

`2> / 2>&1`

SYNOPSIS

```
command 2> error.log
command > all.log 2>&1
```

`2>` 打开文件并把 FD 2 指向它；`2>&1` 不打开文件，而是复制 FD 1 在该时刻的端点。

重要参数 / 形式

`2> error.log / 2>> error.log`

覆盖或追加 stderr。

`> all.log 2>&1`

先改变 FD 1，再让 FD 2 复制它，两个流进入同一文件。

`2>&1 > out.log`

FD 2 先复制原 stdout，随后仅 FD 1 改向文件；两者不等价。

...

... {[.quickref-entry](#)}

管道 |

SYNOPSIS

```
producer | consumer
producer 2>&1 | consumer
```

普通管道把左侧 stdout 连接到右侧 stdin。stderr 默认旁路；需要合流时显式复制 FD 2。

重要参数 / 形式

`A | B`

A 的 FD 1 → 管道 → B 的 FD 0。

```
A 2>&1 | B
```

A 的 stdout 与 stderr 一起进入 B。

```
A |& B
```

Bash 简写；正文优先显式形式以保留 FD 推理。

:::

::: {[.quickref-entry](#)}

tee

SYNOPSIS

```
tee [OPTION]... [FILE]...
```

从 stdin 读取数据，同时写入指定文件和自身 stdout，使数据可以留存后继续进入下游。

重要参数 / 形式

```
tee evidence.log
```

覆盖写入证据文件并继续输出。

```
-a
```

追加到文件，不覆盖既有内容。

```
A | tee before.log | B
```

文件只证明数据到达 B 之前的 `tee` 位置。

:::

::: {[.quickref-entry](#)}

set -o pipefail

SYNOPSIS

```
set -o pipefail
set +o pipefail
```

改变当前 Bash 对整条管道的单一汇总状态计算方式。

重要参数 / 形式

```
set -o pipefail
```

启用；汇总状态取最右侧非零阶段。

```
set +o pipefail
```

关闭并恢复默认“只看最后阶段”的规则。

作用范围

属于当前 Shell 会话状态；临时调查结束后应按原状态恢复。

...

... {[.quickref-entry](#)}

\$? / PIPESTATUS

SYNOPSIS

```
pipeline_rc=$?  
stage_rc=("${PIPESTATUS[@]}")
```

\$? 保存最近命令或管道的单一汇总状态，**PIPESTATUS** 保存最近前台管道的逐阶段状态。

重要参数 / 形式

```
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")  
    在同一个赋值命令中立即复制上一条管道的两类状态。
```

```
${stage_rc[*]}  
    后续展示已保存数组；不要直接展示原 PIPESTATUS 后再声称它仍代表目标管道。
```

...

...

[知识专题] 从终端表象回到文件描述符模型

前面的概念块已经给出对象轮廓，本专题继续把它展开成可操作的路由模型。最常见的误区是把 stdout 等同于屏幕、把 stdin 等同于键盘；更准确的判断是：Shell 在命令启动前建立描述符映射，命令只通过编号读写，终端、普通文件、管道和 `/dev/null` 只是可能的端点。

① [知识点] 0、1、2 是默认文件描述符

一个普通命令启动时通常至少继承三个描述符：

描述符	通道	典型用途	交互式 Shell 中的通常端点
0	stdin	程序读取的输入	当前终端
1	stdout	正常结果、可继续处理的数据	当前终端
2	stderr	诊断、警告和错误信息	当前终端

“通常端点”不是永久绑定。以下三条命令都让 `cat` 输出相同内容，但输入来源不同：

```
cat input.txt
cat < input.txt
printf '%s\n' 'payload' | cat
```

第一条由 `cat` 自己打开文件；第二条由 Shell 先把 FD 0 指向文件；第三条由 Shell 把上游 stdout 和 `cat` 的 stdin 连接起来。

② [知识点] 文件描述符编号与端点必须分开理解

可以把描述符理解为进程内部的一张小型路由表：

```
FD 0 —> 当前输入端点
FD 1 —> 当前正常输出端点
FD 2 —> 当前诊断输出端点
```

端点可以是：

- 控制终端；
- 普通文件；
- 管道的读端或写端；
- `/dev/null`；
- 本章不展开的其他打开对象。

因此，1 不是“屏幕”的代号，2 也不是“红色错误文字”的代号。颜色由终端或程序决定，Shell 只处理数据通道。

③ [知识点] stdout、stderr 与退出状态不是同一类证据

以下命令故意同时产生正常输出、错误输出和非零退出状态：

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "WARN:beta" >&2
  exit 7
'
```

这三个事实应分别判断：

```
stdout 中有什么
stderr 中有什么
命令退出状态是什么
```

程序完全可以向 stderr 写一条警告后返回 0；也可以没有输出任何错误文字却返回非零状态。排错和考试验收都不能只看文字是否“像错误”。

④ [知识点] Shell 在命令运行前准备描述符

重定向不是命令执行完成后的文本搬运。Shell 解析命令行后，先打开目标、复制描述符或建立管道，再启动命令。由此得到两个重要结论：

1. `>` 可能在目标命令真正运行前就截断文件；
2. 目标文件无法打开时，命令主体通常不会开始执行。

这也是为什么“命令失败”与“文件没有变化”不能画等号。

[Cheatsheet] `0/1/2` 是描述符编号；终端、文件和管道是端点；数据通道与退出状态必须分别取证；重定向由 Shell 在命令启动前建立。

[操作专题] 用 `printf`、`cat` 和 `bash` 构造可判定的输入输出

真实命令常同时受权限、语言环境、目录内容和服务状态影响。为了把 FD 关系单独看清，本专题先用可控的生产者与消费者，让每条数据的通道和退出状态都可判定；这里不追求复杂业务，只建立后续所有验证的最小实验。

① [操作] 使用 `printf` 产生确定的 stdout

作用对象：当前命令的 FD 1。基本语义：按格式写出数据；默认写入 stdout。基本形式：

```
printf '%s\n' 'DATA:alpha'
```

建议始终显式写格式字符串。与不同行为实现的 `echo` 相比，`printf` 更适合构造包含反斜杠、连字符或空字符串的确定测试数据。

② [操作] 使用 `>&2` 构造确定的 stderr

```
printf '%s\n' 'WARN:beta' >&2
```

这里 `>&2` 让该次 `printf` 的 stdout 复制 FD 2 当前指向的端点。结果是文字沿 stderr 路径离开命令。该语法也再次说明：重定向作用的是描述符，而不是字符串的“类型”。

③ [操作] 使用 `cat` 观察 stdin 到 stdout 的桥接

作用对象：文件参数或 FD 0 中的数据。基本语义：按顺序读取输入并写向 stdout。典型形式：

```
cat evidence.txt
cat < evidence.txt
printf '%s\n' 'payload' | cat
```

本章只使用 `cat` 作为透明读取与验证入口。行号、不可见字符显示等能力留给后续文本处理内容。

④ [操作] 使用 `bash -c` 同时控制两个流和退出状态

```
bash -c '  
  printf "%s\n" "DATA:alpha"  
  printf "%s\n" "WARN:beta" >&2  
  exit 7  
'
```

这个受控命令适合验证：

- `stdout` 是否进入预期文件；
- `stderr` 是否仍出现在终端；
- 合并顺序是否正确；
- 保存的退出状态是否仍为 `7`。

验证边界：这些命令用于建立可判定的关系；在实际系统中仍应运行命令并保存真实输出、文件内容和退出状态。

[Cheatsheet] 用 `printf` 生产明确数据，用 `>&2` 选择 `stderr`，用 `cat` 验证文件或 `stdin`，用 `bash -c '...; exit N'` 构造可控状态。

[操作专题] 覆盖、追加与分别保存 `stdout/stderr`

重定向文件时，真正需要判断的不是符号长什么样，而是“哪个描述符以什么方式打开哪个目标”。覆盖和追加的差异发生在打开文件时；`stdout` 和 `stderr` 是否分开，则由描述符编号决定。

① [操作] 使用 `>` 覆盖 `stdout` 目标

```
printf '%s\n' 'new data' > output.log
```

`>` 等价于显式写法 `1>`。若目标不存在，Shell 尝试创建；若目标存在，通常先截断为零长度，再把 `FD 1` 指向它。

操作后至少验证两层：

```
cat output.log  
printf 'rc=%s\n' "$?"
```

注意：第二条得到的是 `cat` 的状态，不再是最初 `printf` 的状态。需要保留原命令状态时，必须紧跟在原命令后赋值：

```
printf '%s\n' 'new data' > output.log
command_rc=$?
cat output.log
printf 'command_rc=%s\n' "$command_rc"
```

② [操作] 使用 `>>` 追加 stdout

```
printf '%s\n' 'next data' >> output.log
```

`>>` 让写入发生在文件末尾，不清空已有内容。追加不会自动去重，也不会补救上一行缺少换行的情况。验证时应检查旧证据是否仍存在，而不只看新增内容。

③ [操作] 使用 `2>` 与 `2>>` 单独保存 stderr

```
bash -c 'printf "DATA\n"; printf "WARN\n" >&2; exit 7' \
  > stdout.log 2> stderr.log
```

此时：

```
FD 1 → stdout.log (覆盖)
FD 2 → stderr.log (覆盖)
```

追加错误证据时使用：

```
command 2>> stderr.log
```

不要把 `2>` 读成“第二次重定向”。数字 `2` 明确选择 stderr。

④ [操作] 同时选择输入和输出目标

```
cat < input.txt > output.txt
```

FD 0 和 FD 1 分别指向两个文件。只要路径不同，这种“从文件读取、写入另一文件”的模型很清楚。

高风险反例：

```
cat data.txt > data.txt
```

Shell 可能先截断 `data.txt`，`cat` 随后读到的已经是空文件。不能把普通覆盖重定向当作安全的原地编辑工具。

⑤ [边界] `/dev/null` 会丢弃证据

```
command 2> /dev/null
```

这可以在已经明确错误无关时减少噪声，但不应成为未知故障的默认答案。考试或调查阶段若仍需证明错误类型，应先保存或观察 `stderr`，而不是直接丢弃。

⑥ [边界] `noclobber` 可能改变覆盖行为

某些 Shell 会话可能启用 `noclobber`，使普通 `>` 拒绝覆盖既有文件。遇到“路径可写但覆盖失败”时，应先查询当前 Shell 选项，不要直接删除文件或改权限。本章不把 `noclobber` 展开为独立配置专题。

[Cheatsheet] `> / 1>` 覆盖 `stdout`；`>> / 1>>` 追加 `stdout`；`2>` 覆盖 `stderr`；`2>>` 追加 `stderr`；保存原命令 `$?` 必须发生在下一条验证命令之前。

[知识专题] `2>&1` 的本质是复制当前端点，顺序决定结果

`2>&1` 是本章最容易被错误记忆的语法。它不是“把 2 写进 1”，也不是“将错误写到名为 1 的文件”。它表示：让 FD 2 复制此刻 FD 1 的端点。因为重定向按从左到右应用，FD 1 在该位置指向哪里，会直接决定结果。

① [知识点] `2>` 与 `2>&1` 完成不同动作

```
2> error.log : 打开 error.log, 让 FD 2 指向该文件
2>&1          : 复制 FD 1 当前端点, 让 FD 2 指向同一端点
```

`>` 后面通常是路径；`>&` 明确表示描述符复制。两者不能混为“都是错误重定向”。

② [知识点] `> all.log 2>&1` 将两个流都送入文件

按顺序展开：

```
初始: FD 1 → 终端, FD 2 → 终端
> all.log: FD 1 → all.log, FD 2 → 终端
2>&1: FD 2 复制 FD 1 当前端点
最终: FD 1 → all.log, FD 2 → all.log
```

典型形式：

```
command > all.log 2>&1
```

③ [知识点] `2>&1 > out.log` 只把 `stdout` 送入文件

初始: FD 1 → 终端, FD 2 → 终端
2>&1: FD 2 复制 FD 1 当前端点, 即终端
> out.log: FD 1 → out.log, FD 2 仍指向原终端
最终: FD 1 → out.log, FD 2 → 终端

因此:

```
command 2>&1 > out.log
```

与上一种写法不等价。重定向不存在“Shell 自动帮你理解最终意图”的步骤。

④ [操作] 用受控命令验证顺序

```
bash -c '  
  printf "%s\n" "DATA:alpha"  
  printf "%s\n" "WARN:beta" >&2  
  exit 7  
' > combined-a.log 2>&1  
rc_a=$?
```

另一种顺序:

```
bash -c '  
  printf "%s\n" "DATA:alpha"  
  printf "%s\n" "WARN:beta" >&2  
  exit 7  
' 2>&1 > combined-b.log  
rc_b=$?
```

验收应同时检查: 文件内容、终端仍可见的流, 以及保存的 `rc_a`、`rc_b`。

⑤ [边界] Bash 合并简写不是基础模型的替代品

Bash 支持:

```
command &> all.log  
command &>> all.log
```

它们分别表示合并覆盖和合并追加。为了清晰展示 FD 关系, 并兼顾其他 Bourne 风格 Shell, 本章参考答案优先使用:

```
command > all.log 2>&1  
command >> all.log 2>&1
```

[Cheatsheet] `2>&1` 复制的是“此刻的 FD 1”；从左到右追踪端点；`>file 2>&1` 与 `2>&1 >file` 不等价。

[知识专题] 管道连接的是 stdout 与 stdin，不是命令参数

管道把相邻进程连接成数据流。左侧命令不需要知道右侧是谁，右侧也不需要把左侧输出解析成命令行参数。Shell 建立一个管道对象，让左侧 FD 1 指向写端、右侧 FD 0 指向读端。

① [知识点] A | B 的默认连接关系

A 的 FD 1 → 管道写端
管道读端 → B 的 FD 0

因此：

```
printf '%s\n' 'payload' | cat
```

`payload` 是 `cat` 从 `stdin` 读取的数据，不是 `cat` 的文件名参数。

② [知识点] stderr 默认绕过普通管道

```
bash -c 'printf "DATA\n"; printf "WARN\n" >&2' | cat
```

默认只有 `DATA` 沿管道进入 `cat`；`WARN` 继续沿左侧命令原来的 FD 2 端点输出。普通 `|` 不会自动“收集所有屏幕文字”。

③ [操作] 显式让 stderr 进入管道

```
producer 2>&1 | consumer
```

Bash 也提供简写：

```
producer |& consumer
```

为了让重定向顺序清晰，本章主要使用显式形式 `2>&1 |`。只有在确认运行环境是 Bash 且可读性更好时，才使用 `|&`。

④ [知识点] Shell 先建立管道，再应用每个命令自身的重定向

```
producer > saved.txt | consumer
```

管道原本计划连接 `producer` 的 `stdout`，但 `> saved.txt` 随后把该命令的 `FD 1` 改向文件。结果是数据进入 `saved.txt`，`consumer` 通常收不到该数据。

这不是管道“失效”，而是后应用的命令级重定向覆盖了端点。

⑤ [知识点] 每个管道阶段都是独立的执行对象

以下管道可能把数据正确写入 `evidence.log`，同时中间阶段返回非零：

```
printf '%s\n' 'payload' |  
  bash -c 'cat; exit 9' |  
  tee evidence.log
```

数据路径和状态路径必须分别判断。只看文件或终端内容，无法证明中间阶段成功。

⑥ [边界] 管道是流，不保证完整记录边界

管道传递字节流。程序是否按行读取、何时缓冲、是否提前关闭读端由各程序决定。RHCSA 级操作通常使用行文本工具，但不能把“每行一定立即到达”当成管道本身的保证。

[Cheatsheet] | 默认连接左 `stdout` 到右 `stdin`；`stderr` 旁路；`2>&1` | 显式合流；命令自身重定向可以覆盖管道端点；管道数据可见不等于各阶段成功。

[操作专题] 用 `tee` 在流经位置保留证据

普通 `>` 会把 `stdout` 从后续管道改向文件。`tee` 则作为一个真实的管道阶段读取 `stdin`，把相同数据同时写入一个或多个文件和自己的 `stdout`，因此可以在不截断后续处理链的情况下留下证据。

① [操作] 覆盖写入证据文件并继续输出

```
producer | tee evidence.log | consumer
```

作用对象：`tee` 收到的 `stdin`。基本语义：将输入复制到文件，并原样写到 `stdout`。验证：

```
cat evidence.log
```

该文件证明数据到达了 `tee` 所在位置，但不证明上游和下游全部成功。

② [操作] 使用 `-a` 追加证据

```
producer | tee -a evidence.log | consumer
```

不带 `-a` 时，指定文件会按覆盖方式打开；带 `-a` 时追加。长期调查前应明确是否需要保留历史，避免无意覆盖旧证据或把多次实验混在一起。

③ [知识点] `tee` 的位置定义证据含义

```
A | tee before-B.log | B | tee after-B.log | C
```

- `before-B.log`：B 处理前的数据；
- `after-B.log`：B 处理后的 stdout；
- 两个文件不同，可能说明 B 修改、筛选或未输出数据；
- 两个文件相同，也不能单独证明 B 的退出状态为零。

④ [操作] 同时观察终端和保存文件

当 `tee` 位于管道末端：

```
producer | tee evidence.log
```

stdout 继续连接终端，因此终端可见和文件留存可以同时获得。若只想保存、不想终端显示，仍可在 `tee` 后重定向其 stdout；但这样会改变证据可见性，应写明目的。

⑤ [诊断] 文件没有写入时先检查 `tee` 自身

症状：上游似乎有输出，终端也可能显示内容，但证据文件缺失或不完整。

调查链：

```
确认数据是否真的到达 tee
→ 检查 tee 对应的管道阶段状态
→ 检查路径、父目录、权限和可用空间
→ 用 cat 验证实际文件
→ 修复后重新执行并再次检查状态
```

不要因为 `tee` 是“辅助命令”就忽略它的失败。

[Cheatsheet] `tee file` 覆盖，`tee -a file` 追加；文件证明数据到达 `tee` 的位置；`tee` 是管道阶段，也有独立退出状态。

[操作专题] 用 `$?` 、`PIPESTATUS` 和 `pipefail` 判断管道失败

管道的各阶段可以独立成功或失败，但 Shell 还必须给整条管道提供一个汇总状态。Bash 默认使用最后阶段的状态，这使“上游失败、下游仍正常退出”的管道看起来成功。正确调查需要同时保存汇总状态和逐阶段状态。

① [知识点] 默认管道状态来自最后阶段

```
printf '%s\n' 'payload' |  
  bash -c 'cat; exit 9' |  
  tee evidence.log
```

在默认 Bash 规则下，管道 `$?` 反映最后一个 `tee` 阶段，而不是中间 `bash` 阶段。若 `tee` 成功，汇总状态可能是零。

② [操作] 使用 `${PIPESTATUS[@]}` 查看每个阶段

Bash 的 `PIPESTATUS` 数组按从左到右顺序保存最近一个前台管道各阶段的状态：

```
PIPESTATUS[0] → printf  
PIPESTATUS[1] → bash -c  
PIPESTATUS[2] → tee
```

典型查看形式：

```
printf '%s\n' "${PIPESTATUS[@]}"
```

但直接运行这条查看命令本身又会形成新的简单命令，随后数组会被更新。因此更稳妥的做法是立即复制。

③ [操作] 在一个赋值命令中同时保存汇总和逐阶段状态

```
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
```

Bash 在展开赋值右侧时仍能读取上一条管道留下的 `$?` 和 `PIPESTATUS`，随后把它们保存到普通变量。之后再执行展示命令：

```
printf 'pipeline_rc=%s\n' "$pipeline_rc"  
printf 'stage_rc=%s\n' "${stage_rc[*]}"
```

不要在保存前先运行 `cat`、`echo`、`printf`、`test` 或其他命令。

④ [操作] 启用 `pipefail` 改变汇总规则

```
set -o pipefail
```

启用后：

- 所有阶段都返回零，管道状态为零；
- 有阶段非零，管道状态取最右侧非零阶段的状态。

关闭：

```
set +o pipefail
```

`pipefail` 是当前 Shell 的行为状态。临时调查应在受控范围开启并在结束后恢复，避免无意改变后续脚本或交互命令的判断。

⑤ [知识点] `pipefail` 与 `PIPESTATUS` 解决不同问题

工具	回答的问题
<code>\$?</code>	Shell 对最近命令或管道的单一汇总判断是什么
<code>PIPESTATUS</code>	最近管道的每个阶段分别是什么状态
<code>pipefail</code>	管道汇总状态是否应该暴露中间或上游的非零结果

`pipefail` 不能告诉你所有失败阶段，也不会保存输出数据；`PIPESTATUS` 也不会自动让脚本停止。两者不能互相替代。

⑥ [诊断] 管道看起来成功时的证据链

症状：结果文件有数据，`$?` 也是 `0`
→ 假设：最后阶段成功掩盖了上游失败
→ 下一条证据：立即保存 `PIPESTATUS`
→ 最小修复：在需要可靠汇总的范围启用 `pipefail`
→ 再验证：比较汇总状态、逐阶段状态和数据文件

⑦ [边界] 不把 `set -e` 混入本章结论

`set -e` 与管道、条件命令和函数存在更复杂的交互。本章只训练 `pipefail` 的状态汇总语义，不把“开启 `set -euo pipefail`”当作所有脚本的无条件默认答案。完整脚本错误处理留给 Shell 脚本章节。

[Cheatsheet] 默认 `$?` 看最后阶段；`PIPESTATUS` 看所有阶段；状态必须立即复制；`pipefail` 取最右侧非零；数据证据和状态证据仍要分别验证。

[操作专题] 用 here-document 提供必要的多行标准输入

当命令需要多行 stdin 时，here-document 可以让 Shell 从当前脚本或命令块中收集文本，并把结果连接到命令的 FD 0。它属于输入重定向，而不是临时文件语法或注释语法。

① [操作] 基本 `<<DELIMITER` 形式

```
cat <<EOF
line one
line two
EOF
```

Shell 持续读取，直到遇到只包含结束词的行。开始和结束词应保持一致，结束词不要附带多余空格。

② [知识点] 未引用结束词时正文可能发生展开

```
name='student'
cat <<EOF
user=$name
home=$HOME
EOF
```

未引用 delimiter 时，变量展开、命令替换和算术展开等可能发生。各类引用与展开的完整规则属于前一章《Shell 解析、引用、展开与命令组合》；本章只说明它们会影响最终送入 stdin 的文本。

③ [操作] 引用结束词以保留正文原样

```
cat <<'EOF'
user=$name
host=$(hostname)
EOF
```

引用开始处的 delimiter 会抑制 here-document 正文中的相关展开。引号只写在开始 delimiter 上，结束行仍写裸文本 EOF。

④ [边界] `<<-` 只剥离前导 TAB

```
cat <<-EOF
    line indented with a TAB
EOF
```

`<<-` 便于在脚本中用 TAB 缩进正文和结束词，但不会剥离普通空格。编辑器把 TAB 自动替换为空格时，结束词可能无法按预期匹配。

⑤ [验证] 把 here-document 当作 stdin 检查

```
cat > evidence.txt <<'EOF'
$HOME
$(hostname)
EOF
cat evidence.txt
```

验收重点是文件中是否保留字面量，而不是“命令是否打印过内容”。

⑥ [边界] 不在本章扩展其他输入结构

here-string `<<<`、进程替换 `<(...)`、命名管道和协进程不属于本章必要范围。遇到这些语法时只需识别其不属于普通 here-document，不在此展开。

[Cheatsheet] `<<EOF` 提供多行 stdin；未引用 delimiter 可能展开；`<<'EOF'` 保留字面量；`<<-` 只移除前导 TAB。

[诊断专题] 从表象回溯 FD 路由、数据证据和阶段状态

输入输出故障通常不是“某个符号记错”这么简单。有效诊断应先确定症状属于数据路由、文件打开、管道拓扑还是退出状态，再选择最有区分度的下一条证据。

① [诊断] 终端没有输出

症状：运行命令后终端为空

- 当前证据：只知道终端没有显示
- 假设 A: stdout 被重定向到文件
- 假设 B: stdout 进入管道，下游没有再输出
- 假设 C: 命令没有产生 stdout，只产生 stderr 或无数据
- 下一证据：逐项标注 FD 1/2 端点，检查目标文件和下游状态
- 最小修复：改正端点或在需要的位置加入 tee
- 再验证：终端、文件、退出状态分别检查

“终端为空”不能直接推出命令没有执行。

② [诊断] 错误仍显示在终端

常见原因：

- 只重定向了 stdout；
- `2>&1` 出现时 FD 1 仍指向终端；
- 错误由管道中另一个阶段产生；
- 目标重定向本身打开失败，Shell 把诊断写到当前 stderr。

下一条最有区分度的证据不是继续加更多 `2>`，而是从左到右画出每个命令自己的 FD 映射。

③ [诊断] 命令失败后目标文件被清空

症状：命令没有得到预期结果，但旧文件内容消失

- 当前证据：目标使用了 `>`
- 假设：Shell 在命令主体运行前已截断目标
- 下一证据：检查目标是否同时作为输入、命令主体是否真正启动
- 最小修复：使用不同输出路径，验证后再替换；需要历史时使用明确追加
- 再验证：先检查输出文件，再检查原始输入是否仍完整

不应通过无调查地修改权限、删除原文件或使用强制选项解决。

④ [诊断] 管道 `$?` 为零但业务结果不可信

症状：最后文件存在，`$?` 为 `0`，但怀疑前面阶段失败

- 当前证据：默认汇总只代表最后阶段
- 下一证据：重新执行并立即保存 `PIPESTATUS`
- 最小修复：在该调查范围启用 `pipefail`
- 再验证：数据文件 + `pipeline_rc` + `stage_rc`

若原始执行已经过去且未保存状态，不能从现有文件反推出每个阶段的真实退出值，只能在可重现条件下重新取证。

⑤ [诊断] `tee` 文件为空或缺失

优先区分：

上游没有数据
`tee` 没有运行
`tee` 无法打开目标
`tee` 写入过程中失败
下游状态与 `tee` 文件无关

最小证据组合：

```
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
cat evidence.log
```

再结合目标目录权限和空间进行下一层调查。

⑥ [诊断] here-document 没有结束或内容被意外替换

- 一直等待输入：检查结束词拼写、缩进和尾随空格；
- `$VAR` 被替换：检查开始 delimiter 是否未引用；
- 使用 `<<-` 仍无法匹配：检查缩进是否为普通空格而非 TAB；
- 文本内含结束词：选择更独特的 delimiter。

[Cheatsheet] 先问“哪个 FD 指向哪里”，再查文件和阶段状态；不要用终端表象代替数据证据，不要用最后阶段状态代替整条管道。

[知识专题] 工作迁移：把重定向变成可审计的调查方法

在真实服务器中，重定向常用于收集证据、生成报告输入、保存批处理结果或为后续自动化提供稳定文件。高质量操作需要在执行前明确证据目的，在执行后区分内容、状态和持久文件三个层次。

① [工作方法] 先决定证据是否需要保留历史

- 临时替换：使用新的、独立路径；
- 连续调查：使用 `tee -a` 或带时间边界的文件名；
- 仅减少已确认噪声：才考虑 `/dev/null`；
- 不确定错误：优先单独保存 `stderr`。

② [工作方法] 给不同数据流明确命名

```
command > stdout.log 2> stderr.log
```

比把所有内容无条件合并更利于判断“业务数据缺失”还是“诊断信息增加”。需要完整顺序记录时可以合并，但应在任务说明中写明目的。

③ [工作方法] 把状态与证据文件一起交付

一个调查包至少应说明：

```
运行的命令
执行时的工作目录和用户
stdout/stderr 的保存路径
管道汇总状态
各阶段状态（适用时）
文件内容能够证明的范围
未验证的外部终态
```

④ [工作方法] 为后续自动化保留确定性接口

自动化工具通常分别接收 `stdout`、`stderr` 和返回码。手工阶段先建立正确模型，后续才不会把“任务无失败”扩大为“输出内容正确”，也不会因为把错误合并进数据文件而破坏解析。

[Cheatsheet] 变更前确定证据目的；输出和错误尽量分流；复杂管道保存逐阶段状态；交付时说明证据边界。

[经典任务] 分别保存输出和错误，并证明重定向顺序

任务环境

你在 Bash 中工作，当前目录为：

```
/tmp/rhcsa02-lab
```

目录中可能已经存在以下文件，且可能含有旧内容：

```
stdout.log
stderr.log
combined-a.log
combined-b.log
```

使用以下受控命令作为调查对象：

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
'
```

目标终态

1. 第一次执行时，用覆盖方式把 stdout 保存到 `stdout.log`，把 stderr 保存到 `stderr.log`。
2. 在运行任何其他验证命令前，把受控命令的退出状态保存到变量 `command_rc`。
3. 第二次执行时，把 stdout 追加到 `stdout.log`，仍把 stderr 覆盖写入 `stderr.log`。
4. 分别执行以下两种组合，并保存各自退出状态：

```
> combined-a.log 2>&1
2>&1 > combined-b.log
```

5. 使用 `printf` 加标签，再用 `cat` 显示四个文件内容。
6. 用文件描述符状态说明两种组合为什么不同。
7. 不得把 stderr 丢入 `/dev/null`，不得使用输入文件同时覆盖自身，不把 `&>` 作为主要答案。

验收矩阵

验收层	需要证明的终态
stdout 分流	<code>stdout.log</code> 只保存 DATA 行，并能证明第二次为追加
stderr 分流	<code>stderr.log</code> 只保存最近一次执行的 WARN 行
合并 A	<code>combined-a.log</code> 同时接收 DATA 和 WARN
组合 B	<code>combined-b.log</code> 只接收 stdout，stderr 仍沿原终端端点
状态	保存的受控命令状态为其定义的非零值，而非后续 <code>cat</code> 的状态
解释	能从左到右说明 <code>2>&1</code> 复制时 FD 1 的端点
安全	没有因原地覆盖而破坏输入证据

典型错误

- 先运行 `cat`，再读取 `$?`；
- 把 `2>&1` 当成一个文件路径；
- 没有区分覆盖和追加；
- 只看终端，未检查文件；
- 根据“文件有数据”判断命令成功。

[经典任务] 构造可定位任一阶段失败的调查管道

任务环境

在 Bash 中使用以下管道：

```
printf '%s\n' 'payload' |
  bash -c 'cat; exit 9' |
  tee evidence.log
```

三个阶段分别承担不同职责：阶段 1 产生 `payload`；阶段 2 转发输入但故意以状态 `9` 结束；阶段 3 用 `tee` 写入 `evidence.log` 并继续输出。

目标终态

1. 在默认 Shell 行为下执行管道。
2. 不运行其他简单命令，立即在一次赋值中保存 `pipeline_rc=$?` 与 `stage_rc=("${PIPESTATUS[@]}")`。
3. 显示汇总状态和各阶段状态，解释默认汇总为何可能没有暴露阶段 2 的失败。
4. 记录当前 `pipefail` 状态；开启后重新执行、立即保存两类状态，再恢复原状态。

- 5. 使用 `cat evidence.log` 验证数据到达 `tee`。
- 6. 明确写出：文件有 `payload` 能证明什么，不能证明什么。

验收矩阵

验收层	需要证明的终态
数据	<code>payload</code> 到达 <code>tee</code> 所在位置并写入文件
默认汇总	汇总规则只取最后阶段状态
逐阶段	数组位置与三段管道从左到右对应，并暴露中间非零状态
<code>pipefail</code>	启用后汇总状态反映最右侧非零阶段
会话边界	调查完成后没有无意保留改变的 Shell 选项
结论边界	数据落盘不等于整条管道全部成功

典型错误

- 执行 `echo $?` 后才查看 `PIPESTATUS`；
- 把 `PIPESTATUS` 当成单一数字；
- 认为 `pipefail` 会列出所有失败阶段；
- 开启 `pipefail` 后不恢复会话状态；
- 仅用 `cat evidence.log` 完成验收。

[参考解答] 经典任务一：分流、追加和顺序验证

以下命令是一组可重复的推荐验证序列。执行时应保留文件内容与退出状态作为实际证据。

① 建立目录并进入任务位置

```
mkdir -p /tmp/rhcsa02-lab
cd /tmp/rhcsa02-lab
```

② 覆盖方式分别保存两个流

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' > stdout.log 2> stderr.log
command_rc=$?
```

此时的目标关系：

```
FD 1 → stdout.log
FD 2 → stderr.log
command_rc → 受控命令的退出状态
```

③ 验证覆盖结果与保存状态

```
printf '%s\n' '--- stdout.log ---'
cat stdout.log
printf '%s\n' '--- stderr.log ---'
cat stderr.log
printf 'command_rc=%s\n' "$command_rc"
```

预期关系：stdout 文件只含两条 **DATA**，stderr 文件只含两条 **WARN**，保存状态对应 **exit 7**。验收时以实际运行保存的文件内容和退出状态为准。

④ 第二次执行，stdout 追加、stderr 覆盖

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' >> stdout.log 2> stderr.log
append_rc=$?
```

验收重点：stdout.log 保留第一次两条 DATA 并新增两条；stderr.log 因使用 2> 仍只有最近一次两条 WARN。

⑤ 验证合并顺序 A

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' > combined-a.log 2>&1
rc_a=$?
```

展开：

```
> combined-a.log: FD 1 先指向文件
2>&1: FD 2 再复制 FD 1 当前端点
最终两个流都进入 combined-a.log
```

⑥ 验证组合顺序 B

```
bash -c '
    printf "%s\n" "DATA:alpha"
    printf "%s\n" "DATA:omega"
    printf "%s\n" "WARN:beta" >&2
    printf "%s\n" "WARN:gamma" >&2
    exit 7
' 2>&1 > combined-b.log
rc_b=$?
```

展开:

```
2>&1: FD 2 复制当时 FD 1 的终端端点
> combined-b.log: 随后只把 FD 1 改向文件
最终 stdout 进入文件, stderr 仍指向原终端
```

⑦ 分层显示最终证据

```
printf '%s\n' '=== stdout.log ==='
cat stdout.log
printf '%s\n' '=== stderr.log ==='
cat stderr.log
printf '%s\n' '=== combined-a.log ==='
cat combined-a.log
printf '%s\n' '=== combined-b.log ==='
cat combined-b.log
printf 'append_rc=%s rc_a=%s rc_b=%s\n' "$append_rc" "$rc_a" "$rc_b"
```

⑧ 结论边界

- `cat` 成功只能证明文件当前可读, 不能替代之前命令状态;
- `combined-a.log` 中存在 `WARN` 证明 `stderr` 被合并到文件, 不证明命令成功;
- `combined-b.log` 不含 `WARN` 是顺序语义的结果, 不代表命令没有产生 `stderr`。

[参考解答] 经典任务二：保存逐阶段状态并使用 `pipefail`

① 查询并记录初始 `pipefail` 状态

```
if shopt -qo pipefail; then
    pipefail_was_on=true
else
    pipefail_was_on=false
fi
```

上述查询写法较长。考试现场若只需临时开启并在任务结束明确关闭，也可记录自己的操作边界。这里强调的是不要无意污染后续会话。

② 默认行为下执行并立即保存状态

```
set +o pipefail

printf '%s\n' 'payload' |
    bash -c 'cat; exit 9' |
    tee evidence.log
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
```

随后才展示：

```
printf 'pipeline_rc=%s\n' "$pipeline_rc"
printf 'stage_rc=%s\n' "${stage_rc[*]}"
```

按语义，数组从左到右对应三个阶段。默认汇总只取最后阶段，因此即使阶段 2 非零，`pipeline_rc` 仍可能为零。

③ 开启 `pipefail` 后重新执行

```
set -o pipefail

printf '%s\n' 'payload' |
    bash -c 'cat; exit 9' |
    tee evidence.log
pipefail_rc=$? pipefail_stage_rc=("${PIPESTATUS[@]}")
```

展示：

```
printf 'pipefail_rc=%s\n' "$pipefail_rc"
printf 'pipefail_stage_rc=%s\n' "${pipefail_stage_rc[*]}"
```

按 Bash 语义，汇总状态应反映最右侧非零阶段；逐阶段数组仍用于确认具体位置。

④ 恢复初始状态

```
if [ "$pipefail_was_on" = true ]; then
    set -o pipefail
else
    set +o pipefail
fi
```

⑤ 验证数据文件

```
printf '%s\n' '=== evidence.log ==='
cat evidence.log
```

⑥ 分层结论

evidence.log 中存在 payload
→ 证明 payload 到达 tee 且 tee 成功写入该文件

阶段 2 状态非零
→ 证明中间处理阶段按设计失败

默认 pipeline_rc 可能为零
→ 只反映最后阶段成功，不能代表全管道成功

pipefail_rc 为非零
→ 汇总规则暴露了最右侧非零阶段

⑦ 更简洁的考试型写法

若题目只要求在当前受控 Bash 中可靠判断管道，并允许任务后明确关闭：

```
set -o pipefail
pipeline | tee evidence.log
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
set +o pipefail
```

仍需注意：关闭 pipefail 的命令必须在状态已经保存之后执行。

[本章收束] 用同一模型解释重定向、管道和证据

完成本章后，面对任何输入输出命令，不应先猜“该加哪个符号”，而应按固定顺序还原路由和证据：

1. 标出命令的 FD 0、1、2；

- 2. 从左到右应用重定向，记录每一步端点变化；
- 3. 对管道逐段确认“左 stdout → 右 stdin”，并单独追踪 stderr；
- 4. 判断数据在哪个位置仍可观察、保存或被丢弃；
- 5. 在任何新命令之前保存退出状态；
- 6. 对复杂管道同时保留汇总状态和 PIPESTATUS；
- 7. 写清楚每份证据能证明什么、不能证明什么。

⋮ {method-box} 工作方法：先画路由，再读结果。只要能把每个阶段写成“FD 编号 → 当前端点”，>、2>&1、管道和 tee 就不再是孤立符号；只要把数据证据和退出状态分开，管道“看起来成功”的误判也会明显减少。⋮

主要判断表

需求或现象	首选形式或下一条证据	证明边界
从文件提供 stdin	<code>command < input</code>	只证明 FD 0 来源
覆盖或追加 stdout	<code>> / >></code>	覆盖可能先截断；追加不自动去重
单独保存或合并 stderr	<code>2> error / > all 2>&1</code>	仍需单独保存退出状态
只把 stdout 送入下游	<code>A B</code>	stderr 默认旁路
把两个流送入下游	<code>A 2>&1 B</code>	合流后下游无法区分原始通道
保存并继续传递	<code>A tee [-a] file B</code>	文件只证明数据到达 tee 位置
保存汇总与逐阶段状态	<code>rc=\$? + PIPESTATUS</code>	必须在任何后续简单命令前复制
暴露中间失败	<code>set -o pipefail</code>	仍不能指出全部失败阶段
多行 stdin	<code><<EOF / <<'EOF'</code>	delimiter 引用决定正文是否展开

向下一章交接

本章已经回答数据从哪里进入命令、stdout/stderr 去哪里、重定向何时生效、管道如何连接进程，以及怎样判断中间阶段失败。下一章《本地帮助、命令发现与软件能力查询》将说明怎样定位 printf、cat、tee 和 bash 的真实入口、synopsis、手册与软件包来源。第 05 章再展开 grep、sed、awk 等管道消费者，第 13 章才完整处理 Journal、rsyslog 和日志轮转。